

# 北京邮电大学

## 《大数据安全》作业



题目：

主笔人：班

学院：网络空间安全学院

协助

班

班

班

班

# FIDO 协议：为网站添加用户管理功能，支持 FIDO2 认证

## 一、FIDO 协议基础

FIDO (Fast IDentity Online, 快速在线身份认证) 协议是一种开放标准, 旨在实现更加安全、便捷的无密码身份验证机制, 减少传统口令认证带来的风险, 如密码泄露、重放攻击及钓鱼攻击等问题。该标准由 FIDO 联盟 (FIDO Alliance) 推动, 目前主要包括三种规范: FIDO UAF、FIDO U2F 与 FIDO2。FIDO UAF (Universal Authentication Framework)

### (1) FIDO UAF (通用认证框架)

FIDO UAF 支持用户通过设备上的本地生物识别技术 (如指纹、人脸识别等) 进行身份验证, 而无需输入传统密码。在注册阶段, 用户设备会创建一对密钥, 并将私钥安全地保存于本地, 仅将公钥提交到认证服务器。当用户再次登录时, 设备利用私钥生成数字签名, 服务器通过验证该签名来确认用户身份, 从而实现无密码认证。

### (2) FIDO U2F (通用第二因素)

FIDO U2F 主要用于双因素认证 (2FA) 场景, 通常与用户名及密码共同使用。在注册过程中, 用户的安全密钥生成一对密钥, 公钥存储于服务器端。当用户登录时, 在输入密码后, 系统会提示插入硬件密钥, 密钥通过私钥对服务器提供的挑战值进行签名, 从而完成身份验证。这种方式能够显著提升账号安全性。

### (3) FIDO2 (FIDO2 = WebAuthn+CTAP2)

FIDO2 是该系列标准的最新版本, 由 WebAuthn 与 CTAP2 两部分组成, 可同时支持无密码登录与双因素认证。

WebAuthn 是由 W3C 与 FIDO 联盟联合制定的基于公钥加密的 Web 认证标准, 使网站能够调用系统或外部认证设备 (如指纹识别器、USB 安全密钥) 作为登录凭证。

CTAP2 则定义了浏览器/操作系统与认证器 (例如指纹模块、YubiKey 等硬件密钥) 之间的通信方式。借助 FIDO2, 用户可在手机、电脑等多平台上使用安全设备进行登录, 从而摆脱对传统密码的依赖。

## 二、FIDO2 工作流程

FIDO2 的整个身份认证流程分为注册阶段和登录阶段两部分。以下是其详细机制说明。

(1) 注册阶段: 当用户首次访问网站时, 需要完成 FIDO2 凭证注册, 过程如下。

a. 用户发起注册请求: 用户在网站注册界面 (/register) 输入用户名和密码等基础信息, 并请求启用 FIDO2 认证。前端通过表单提交用户名和密码到后端。

b.服务器生成挑战值：网站服务器接收注册请求后，首先验证用户名是否存在。若不存在，使用 `generate_password_hash()` 对密码进行加密存储。然后在 `/fido_register` 路由中，调用 `server.register_begin(user_entity)` 创建一个随机的挑战（challenge），确保认证过程的唯一性与安全性。服务器将挑战值和用户信息（用户 ID、用户名、显示名称）、RP 信息（应用名称和 ID）、支持的公钥算法等打包成注册选项，并将其存储到会话中。

c.调用 WebAuthn API：前端页面接收服务器返回的注册选项后，将 Base64 编码的 challenge 和 user.id 转换为 ArrayBuffer 格式，然后调用 `navigator.credentials.create()` 方法，并将服务器生成的挑战传递给浏览器。

d.生成密钥对：用户的认证设备（如指纹传感器、手机、硬件密钥等）接收该请求，并提示用户进行身份验证（如指纹或 PIN 码）。验证通过后，设备会生成独立的公私钥对。私钥：保存在本地设备，绝不会上传。公钥：将返回服务器用于后续验证。设备生成的凭证包含 `rawId`（凭证 ID）、`response.clientDataJSON`（客户端数据）和 `response.attestationObject`（证明对象）。

e.服务器存储凭证：设备把公钥及相关元信息（如凭证 ID、算法类型等）传回服务器。前端将凭证数据（Base64 编码）发送到 `/fido_complete_register` 路由，服务器调用 `server.register_complete()` 验证凭证数据的有效性。验证成功后，服务器提取凭证 ID 和公钥，使用 CBOR 编码公钥，并将其与用户账户关联保存到 `users[username]["credentials"]` 中。

**（2）登录阶段：**当用户下次访问该网站时，可以直接通过 FIDO2 设备登录，过程如下。

a.发起登录请求：用户在 `/login` 页面输入用户名和密码，并请求使用 FIDO2 登录。前端通过表单提交用户名和密码到后端。

b.生成挑战信息：服务器接收登录请求后，首先使用 `check_password_hash()` 验证用户名和密码是否正确。验证成功后，将用户名存储到会话中。然后在 `/fido_auth` 路由中，服务器为该会话生成新的挑战，从用户账户中读取已保存的凭证 ID，调用 `server.authenticate_begin([credential_descriptor])` 生成认证选项，并将与用户关联的公钥元数据（凭证 ID、challenge、rpId 等）发送到客户端。

c.调用 WebAuthn API：前端调用 `navigator.credentials.get()` 方法，将 Base64 编码的 challenge 和 `allowCredentials` 中的凭证 ID 转换为 ArrayBuffer 格式，将挑战值传递给认证设备。

d.签名生成：认证设备提示用户再次进行身份验证（如指纹识别），验证通过后，使用私钥对挑战进行签名，并返回签名数据。设备返回的断言包含 `rawId`（凭证 ID）、`response.clientDataJSON`（客户端数据）、`response.authenticatorData`（认证器数据）和 `response.signature`（签名）。

e.验证签名：前端将签名数据（Base64 编码）发送到 `/fido_login` 路由。服务器接收签名结果后，从会话中获取之前存储的认证状态，从用户账户中读取已

保存的公钥，调用 `server.authenticate_complete()` 通过已保存的公钥进行验证，确认签名真实有效。若验证成功，即表明用户确实持有对应的私钥，登录被批准。服务器在会话中设置 `authenticated` 标志为 `True`。

这一流程实现了真正的“无密码安全登录”，同时兼顾了跨平台兼容性和强身份保障，使 FIDO2 成为现代网络认证体系的核心标准之一。

### 三、项目实现

项目设计结构图如下：



#### 1. 首先配置环境，确保浏览器和设备支持 FIDO2（WebAuthn）认证功能

```
pip install -r requirements.txt
```

```
requirements.txt
1  Flask==2.3.3
2  Werkzeug==2.3.7
3  fido2==1.1.2
...
```

```
(mypytorch) PS C:\Users\1\Desktop\Bigdata-hw3> pip install -r requirements.txt
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Collecting Flask==2.3.3 (from -r requirements.txt (line 1))
  Using cached https://pypi.tuna.tsinghua.edu.cn/packages/fd/56/26f0be8adc2b4257df20c1c4260ddd0aa396cf8e75d90ab2f7ff99bc34f9/flask-2.3.3-py3-none-any.whl (96 kB)
Collecting Werkzeug==2.3.7 (from -r requirements.txt (line 2))
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/9b/59/a7c32e3d8d0e546a206e0552a2c04444544f15c1da4a01df8938d20c6ffc/werkzeug-2.3.7-py3-none-any.whl (242 kB)
Collecting fido2==1.1.2 (from -r requirements.txt (line 3))
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/e3/99/68bab31fbb49e8b3a58ce31cfde5ed9aede6efe5eebbece2eaf93ade75c0/fido2-1.1.2-py3-none-any.whl (203 kB)
Collecting cryptography==41.0.3 (from -r requirements.txt (line 4))
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/30/56/5f4eee57ccd577c261b516bfcbe17492838e2bc4e2e92ea93bbb57666fbd/cryptography-41.0.3-cp37-abi3-win_amd64.whl (2.6 MB)
  2.6/2.6 MB 7.3 MB/s eta 0:00:00
Requirement already satisfied: Jinja2>=3.1.2 in c:\users\1\.conda\envs\mypytorch\lib\site-packages (from Flask==2.3.3->-r requirements.txt (line 1)) (3.1.4)
Requirement already satisfied: itsdangerous>=2.1.2 in c:\users\1\.conda\envs\mypytorch\lib\site-packages (from Flask==2.3.3->-r requirements.txt (line 1)) (2.2.0)
Requirement already satisfied: click>=8.1.3 in c:\users\1\.conda\envs\mypytorch\lib\site-packages (from Flask==2.3.3->-r requirements.txt (line 1)) (8.2.1)
Requirement already satisfied: blinker>=1.6.2 in c:\users\1\.conda\envs\mypytorch\lib\site-packages (from Flask==2.3.3->-r requirements.txt (line 1)) (1.9.0)
Requirement already satisfied: MarkupSafe>=2.1.1 in c:\users\1\.conda\envs\mypytorch\lib\site-packages (from Werkzeug==2.3.7->-r requirements.txt (line 2)) (2.1.5)
Requirement already satisfied: cffi>=1.12 in c:\users\1\.conda\envs\mypytorch\lib\site-packages (from cryptography==41.0.3->-r requirements.txt (line 4)) (1.17.1)
Requirement already satisfied: pycparser in c:\users\1\.conda\envs\mypytorch\lib\site-packages (from cffi>=1.12->cryptography==41.0.3->-r requirements.txt (line 4)) (2.2.3)
```

```

Requirement already satisfied: pycparser in c:\users\1\.conda\envs\mypytorch\lib\site-packages (from cffi>=1.12->cryptography==41.0.3->-r requirements.txt (line 4)) (2.22)
Requirement already satisfied: colorama in c:\users\1\.conda\envs\mypytorch\lib\site-packages (from click>=8.1.3->Flask==2.3.3->-r requirements.txt (line 1)) (0.4.6)
Installing collected packages: Werkzeug, Flask, cryptography, fido2
  Attempting uninstall: Werkzeug
    Found existing installation: Werkzeug 3.1.3
    Uninstalling Werkzeug-3.1.3:
      Successfully uninstalled Werkzeug-3.1.3
  Attempting uninstall: Flask
    Found existing installation: Flask 3.1.2
    Uninstalling Flask-3.1.2:
      Successfully uninstalled Flask-3.1.2
  Attempting uninstall: cryptography
    Found existing installation: cryptography 45.0.2
    Uninstalling cryptography-45.0.2:
      Successfully uninstalled cryptography-45.0.2
Successfully installed Flask-2.3.3 Werkzeug-2.3.7 cryptography-41.0.3 fido2-1.1.2

```

## 2. 编写程序

(1) 首先在 `app.py` 中导入实现 FIDO2 认证所需的所有模块

```

from flask import Flask, render_template, request, jsonify, session, redirect, url_for
from werkzeug.security import generate_password_hash, check_password_hash
from fido2.server import Fido2Server
from fido2.webauthn import PublicKeyCredentialRpEntity, PublicKeyCredentialUserEntity, PublicKeyCredentialDescriptor
from fido2.utils import websafe_encode, websafe_decode
import os
import json
import base64

```

模块说明：

Flask 相关：用于构建 Web 应用和路由处理

`werkzeug.security`：用于密码加密和验证

`fido2.server.Fido2Server`：实现 WebAuthn 协议的 FIDO2 认证服务，用于生成和验证 FIDO2 注册和认证请求

`fido2.webauthn`：包含 FIDO2 凭证相关的数据结构

`base64`：用于编码/解码 FIDO2 数据

(2) 初始化 Flask 应用和 FIDO2 服务器

```

# 启用 WebAuthn JSON 映射
import fido2.features
fido2.features.webauthn_json_mapping.enabled = True

app = Flask(__name__)
app.secret_key = os.urandom(24)

# 配置 RP Entity (Relying Party)，使用正确的 origin
rp = PublicKeyCredentialRpEntity(id="localhost",
name="Example App")

```

```

# 定义 origin 验证函数
def verify_origin(origin):
    # 允许 localhost 的所有端口
    return origin in ["http://localhost:5000",
"https://localhost:5000"]

# 创建 Fido2Server, 指定 origin 验证函数
server = Fido2Server(rp, attestation="none",
verify_origin=verify_origin)

# 全局用户存储（实际应用应使用数据库）
users = {}

```

配置说明:

RP Entity: 代表依赖方（应用程序），用于标识应用

origin 验证函数: 确保 FIDO2 请求来自正确的源

users 字典: 存储用户信息，包括密码哈希和 FIDO2 凭证

### （3）用户注册界面

后端路由 (/register):

```

@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        username = request.form["username"]
        password = request.form["password"]
        if username in users:
            return "User already exists", 400
        hashed_password = generate_password_hash(password)
        users[username] = {"password": hashed_password,
"credentials": None}
        return redirect(url_for("fido_setup",
username=username))
    return render_template("register.html")

```

前端界面 (register.html):

用户输入用户名和密码

前端验证两次密码输入是否一致

提交表单后，后端存储加密的密码，并重定向到 FIDO2 设置页面

### （4）FIDO2 设置页面

后端路由 (/fido\_setup):

```

@app.route("/fido_setup/<username>", methods=["GET"])
def fido_setup(username):
    if username not in users:
        return "User not found", 404
    return render_template("fido_setup.html",
username=username)

```

前端交互 (fido\_setup.html):

显示 FIDO2 设置步骤指引

用户点击"开始 FIDO2 注册"按钮

前端向/fido\_register 发送请求获取注册选项

(5) 生成 FIDO2 注册选项

后端路由(/fido\_register):

```
@app.route("/fido_register", methods=["POST"])
def fido_register():
    username = request.form.get("username")
    if not username or username not in users:
        return jsonify({"error": "User not found"}), 404

    try:
        user_entity = PublicKeyCredentialUserEntity(
            id=username.encode(),
            name=username,
            display_name=username
        )
        registration_data, state =
server.register_begin(user_entity)

        # 存储状态到会话
        session["fido_registration_state"] = state

        # 将 registration_data 转换为可序列化的格式
        pk = registration_data.public_key
        challenge = base64.b64encode(pk.challenge).decode()
        user_id = base64.b64encode(pk.user.id).decode()

        response_data = {
            "publicKey": {
                "challenge": challenge,
                "rp": {"name": pk.rp.name, "id": pk.rp.id},
                "user": {
                    "id": user_id,
                    "name": pk.user.name,
                    "displayName": pk.user.display_name
                },
            },
            "pubKeyCredParams": [
                {"type": "public-key", "alg": param.alg}
                for param in pk.pub_key_cred_params
            ],
            "timeout": 60000,
            "attestation": "direct"
```

```

    }
}
return jsonify(response_data)
except Exception as e:
    return jsonify({"error": str(e)}), 400

```

前端处理 (fido\_setup.html):

接收注册选项, 将 Base64 编码的数据转换为 ArrayBuffer

调用 navigator.credentials.create() 创建 FIDO2 凭证

用户使用生物识别或硬件密钥完成认证

#### (6) 完成 FIDO2 注册

后端路由 (/fido\_complete\_register):

```

@app.route("/fido_complete_register", methods=["POST"])
def fido_complete_register():
    data = request.get_json()
    username = data.get("username")

    if not username or username not in users:
        return jsonify({"error": "User not found"}), 404

    try:
        state = session.pop("fido_registration_state", None)
        if not state:
            return jsonify({"error": "Registration state not found"}), 400

        from fido2.webauthn import CollectedClientData,
        AttestationObject

        # 解码 Base64 数据
        client_data_bytes =
base64.b64decode(data["response"]["clientDataJSON"])
        attestation_object_bytes =
base64.b64decode(data["response"]["attestationObject"])

        # 创建对象并完成注册
        client_data = CollectedClientData(client_data_bytes)
        attestation_object =
        AttestationObject(attestation_object_bytes)
        auth_data = server.register_complete(state,
        client_data, attestation_object)

        # 存储凭证数据
        from fido2 import cbor

```



```

        cred_id =
base64.b64encode(auth_data.credential_data.credential_id).dec
ode()

        public_key = auth_data.credential_data.public_key
        public_key_bytes = cbor.encode(public_key)

        users[username]["credentials"] = json.dumps({
            "credential_id": cred_id,
            "public_key":
base64.b64encode(public_key_bytes).decode()
        })

        return jsonify({"status": "registered"})
    except Exception as e:
        return jsonify({"error": str(e)}), 400

```

### (7) 用户登录

后端路由(/login):

```

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form["username"]
        password = request.form["password"]
        if username not in users or not
check_password_hash(users[username]["password"], password):
            return "Invalid username or password", 400
        session["username"] = username
        return redirect(url_for("fido_auth"))
    return render_template("login.html")

```

前端交互 (login.html):

用户输入用户名和密码

前端提交表单到后端验证

验证成功后，自动请求 FIDO2 认证选项

### (8) FIDO2 认证

后端路由 (/fido\_auth):

```

@app.route("/fido_auth", methods=["GET"])
def fido_auth():
    username = session.get("username")
    if not username or username not in users:
        return "Unauthorized", 401

    if users[username].get("credentials") is None:
        return "User has no FIDO2 credentials", 400

    try:

```

```

        cred_data =
json.loads(users[username]["credentials"])
        credential_id =
base64.b64decode(cred_data["credential_id"])

        credential_descriptor =
PublicKeyCredentialDescriptor(
            type="public-key",
            id=credential_id
        )

        authentication_data, state =
server.authenticate_begin([credential_descriptor])
        session["fido_authentication_state"] = state

        pk = authentication_data.public_key
        challenge = base64.b64encode(pk.challenge).decode()
        cred_id_encoded =
base64.b64encode(credential_id).decode()

        response_data = {
            "publicKey": {
                "challenge": challenge,
                "timeout": 60000,
                "rpId": pk.rp_id,
                "userVerification": "preferred",
                "allowCredentials": [
                    {"type": "public-key", "id":
cred_id_encoded}
                ]
            }
        }

        return jsonify(response_data)
    except Exception as e:
        return jsonify({"error": str(e)}), 400

```

#### (9) 完成 FIDO2 登录

后端路由 (/fido\_login):

```

@app.route("/fido_login", methods=["POST"])
def fido_login():
    username = session.get("username")
    if not username or username not in users:
        return jsonify({"error": "Unauthorized"}), 401

```

```

data = request.get_json()
try:
    state = session.pop("fido_authentication_state",
None)
    if not state:
        return jsonify({"error": "Authentication state not
found"}), 400

    cred_data =
json.loads(users[username]["credentials"])
    stored_cred_id =
base64.b64decode(cred_data["credential_id"])

    from fido2 import cbor
    public_key_bytes =
base64.b64decode(cred_data["public_key"])
    public_key = cbor.decode(public_key_bytes)

    from fido2.webauthn import CollectedClientData,
AuthenticatorData, AttestedCredentialData, Aaguid

    # 解码 Base64 数据
    credential_id = base64.b64decode(data["id"])
    client_data_bytes =
base64.b64decode(data["response"]["clientDataJSON"])
    authenticator_data_bytes =
base64.b64decode(data["response"]["authenticatorData"])
    signature =
base64.b64decode(data["response"]["signature"])

    # 创建对象
    client_data = CollectedClientData(client_data_bytes)
    authenticator_data =
AuthenticatorData(authenticator_data_bytes)

    # 重建 AttestedCredentialData 对象用于验证
    attested_cred_data = AttestedCredentialData.create(
        aaguid=Aaguid.NONE,
        credential_id=stored_cred_id,
        public_key=public_key
    )

    # 完成认证
    server.authenticate_complete(

```

```

        state,
        [attested_cred_data],
        credential_id,
        client_data,
        authenticator_data,
        signature
    )

    session["authenticated"] = True
    return jsonify({"status": "authenticated"})
except Exception as e:
    return jsonify({"error": str(e)}), 400

```

(10) 仪表板和登出

后端路由:

```

@app.route("/", methods=["GET"])
def index():
    if session.get("authenticated"):
        return redirect(url_for("dashboard"))
    return render_template("index.html")

@app.route("/dashboard", methods=["GET"])
def dashboard():
    if not session.get("authenticated"):
        return redirect(url_for("login"))
    username = session.get("username")
    return render_template("dashboard.html",
        username=username)

@app.route("/logout", methods=["GET"])
def logout():
    session.clear()
    return redirect(url_for("index"))

```

### 3.最后运行本项目，执行 python app.py

```

(mypytorch) PS C:\Users\1\Desktop\Bigdatasecurity-hw3> & C:/Users/1/.conda/envs/mypytorch/python.exe c:
/Users/1/Desktop/Bigdatasecurity-hw3/app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
server instead.
* Running on http://localhost:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 182-260-740

```

前往 <http://localhost:5000/register> 注册新用户

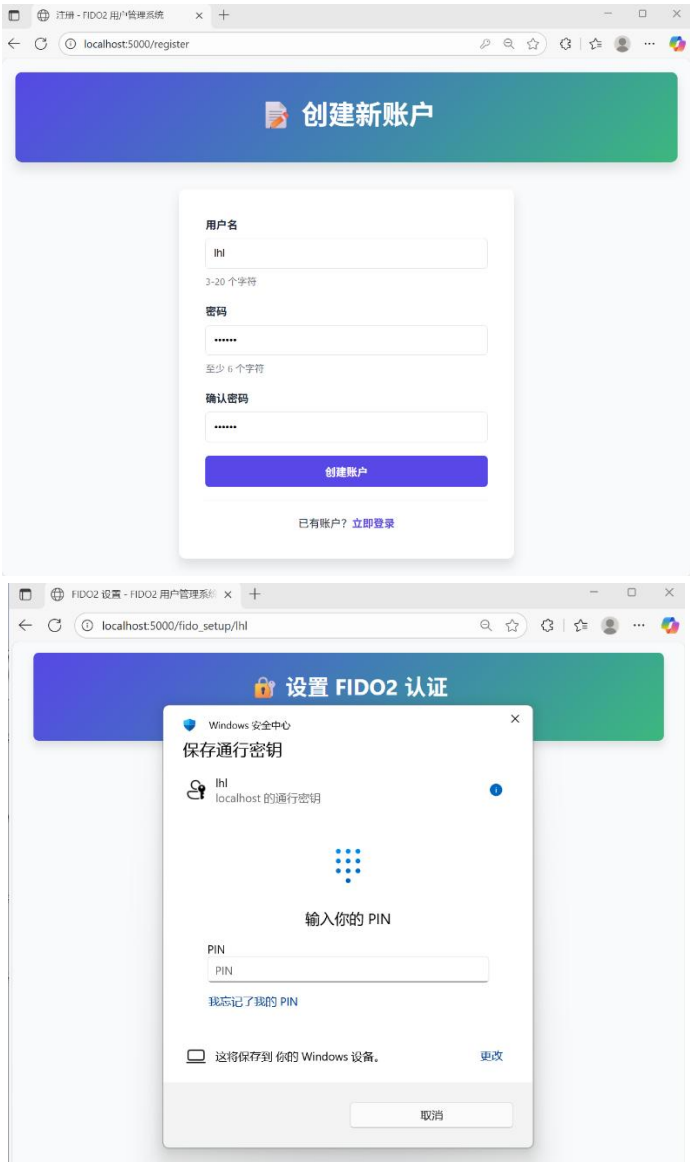
前往 <http://localhost:5000/login> 进行登录

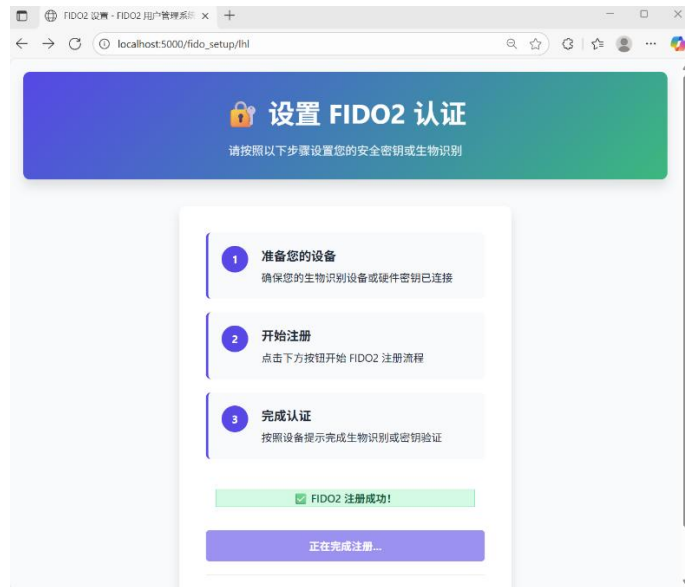
4.以下是界面操作展示

(1) 网站界面

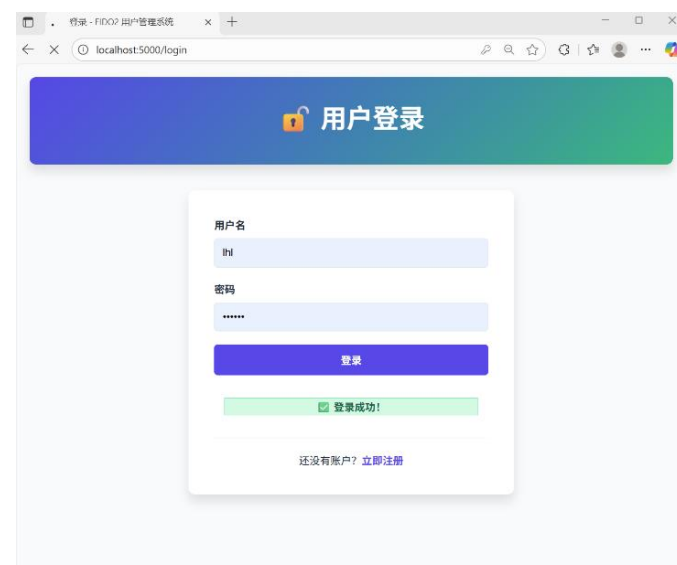
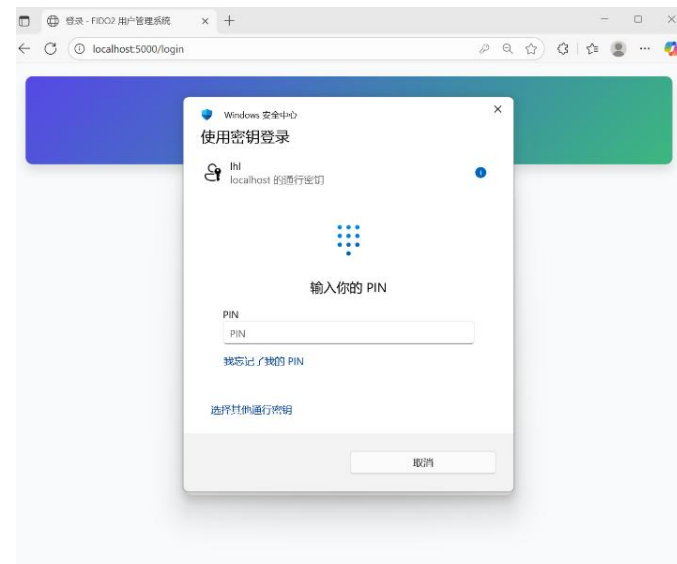


(2) 创建新用户

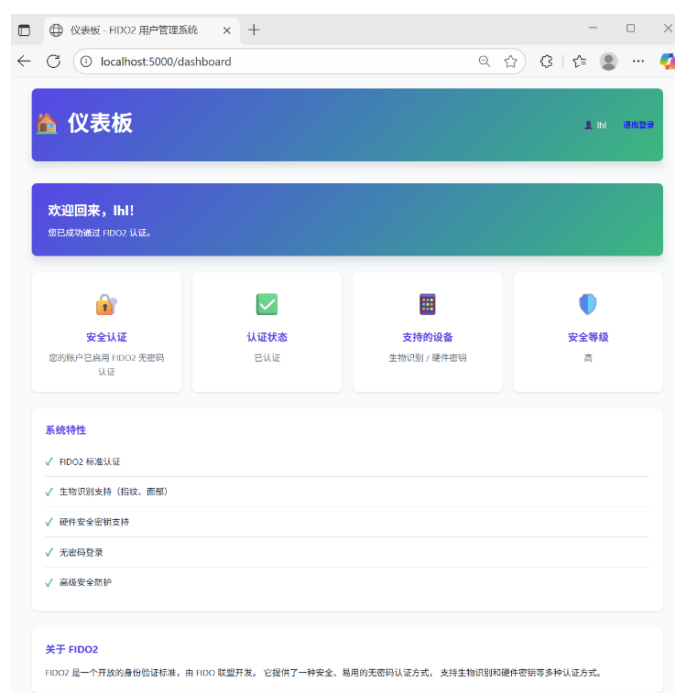




### (3) 用户登录



#### (4) 登录界面



## 四、总结实现原理

**用户注册：**在/register 页面，用户提供用户名和密码进行注册。注册成功后，系统将用户信息（用户名和加密后的密码）存储到内存中，并将用户重定向到 /fido\_setup 页面进行 FIDO2 凭证绑定。

**FIDO2 设置：**在/fido\_setup 路由，用户点击"开始 FIDO2 注册"按钮后，前端向 /fido\_register 路由发送请求。后端生成注册选项（包含 challenge、用户信息、RP 信息等），并将其转换为 JSON 格式返回给前端。前端通过 navigator.credentials.create()调用用户的 FIDO2 设备（如生物识别或硬件密钥）创建凭证。生成的凭证数据（包含 clientDataJSON 和 attestationObject）通过 /fido\_complete\_register 路由发送回服务器，服务器验证并保存凭证信息（credential\_id 和 public\_key）到用户账户中。

**用户登录：**用户在/login 页面输入用户名和密码。后端验证用户名和密码是否正确。若正确，系统将用户名存储到会话中，并将用户重定向到 FIDO2 认证流程。

**FIDO2 验证：**登录表单提交后，前端向/fido\_auth 路由请求认证选项。后端根据存储的凭证信息生成认证选项（包含 challenge 和 allowCredentials），返回给前端。前端调用 navigator.credentials.get()完成 FIDO2 设备验证。验证成功后，生成的凭证数据（包含 clientDataJSON、authenticatorData 和 signature）通过 /fido\_login 路由发送到服务器进行最终验证。服务器验证签名和数据的完整性，若通过，则在会话中设置 authenticated 标志，用户便可访问/dashboard 仪表盘。